

Student Details

Name: R.Varshitha

Roll Number: 24501A4450

Experiment Title: Multiple Linear Regression using Sales Advertising Dataset

Dataset: Sales Advertising Dataset

Algorithm: Multiple Linear Regression

1. Business Understanding

1.1 Problem Statement & Relevance

Advertising plays a significant role in increasing product sales. Companies invest in different advertising channels such as **TV, Radio, and Newspaper** to promote their products and attract customers. Understanding how each advertising medium contributes to sales helps organizations optimize their marketing strategies.

In this experiment, a **Multiple Linear Regression** model is developed to predict **Sales** using **TV, Radio, and Newspaper advertising expenditures** as independent variables.

Problem Statement

Develop a machine learning model that predicts **Sales** based on advertising expenditures on **TV, Radio, and Newspaper**.

```
#import libraries Source Code: import pandas as pd import matplotlib.pyplot as plt import numpy as np import seaborn as sns from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score print("Libraries Imported")
```

Libraries Imported

```
#Load Dataset
```

```
df = pd.read_csv('/content/advertising.csv') print("Dataset reading complete")
```

Output: Dataset reading complete

```
#Display 5 rows
```

```
df.head()
```

Output:

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

#Display the last 5 rows of dataset
df.tail()

Output:

	TV	radio	newspaper	sales
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	9.7
197	177.0	9.3	6.4	12.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	13.4

Source Code:

df.shape

Output:

(200, 4)

Summary of a dataframe

Df.info Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   TV           200 non-null   float64
1   radio        200 non-null   float64
2   newspaper    200 non-null   float64
3   sales        200 non-null   float64
dtypes: float64(4)
memory usage: 6.4 KB
```

Relevance

- Helps businesses forecast future sales.

- Supports effective marketing budget allocation.
- Identifies the impact of each advertising channel.
- Assists managers in making data-driven marketing decisions.
- Useful for business analytics and sales forecasting.

1.2 Expected Benefits

- Accurate prediction of product sales.
- Better allocation of advertising budgets.
- Improved marketing strategies.
- Increased return on advertising investment (ROI).
- Supports business planning and decision-making.

2. Data Understanding

2.1 Dataset Structure

Dataset Name: Sales Advertising Dataset

Number of Records: 200

Number of Attributes: 4

Target Variable

Sales

Define the dependent variable (target) Source
Code:

```
y = df['sales']  
Y
```

Output:

```

      sales
0    22.1
1    10.4
2     9.3
3    18.5
4    12.9
...     ...
195    7.6
196    9.7
197   12.8
198   25.5
199   13.4
200 rows × 1 columns

dtype: float64

```

Independent Variables

- TV
- Radio
- Newspaper

Define the independent variables (features) Source Code:

```
X = df.drop('sales', axis=1)
```

X

Output:

```

      TV  radio  newspaper
0  230.1   37.8     69.2
1   44.5   39.3     45.1
2   17.2   45.9     69.3
3  151.5   41.3     58.5
4  180.8   10.8     58.4
...     ...     ...
195  38.2    3.7     13.8
196  94.2    4.9      8.1
197 177.0    9.3      6.4
198 283.6   42.0     66.2
199 232.1    8.6      8.7
200 rows × 3 columns

```

2.2 Data Dictionary

Column	Description	Data
--------	-------------	------

		Type
TV	Advertising expenditure on Television (in thousands of Float dollars)	
Radio	Advertising expenditure on Radio (in thousands of dollars)	Float
Newspaper	Advertising expenditure on Newspaper (in thousands of Float dollars)	
Sales	Product sales (in thousands of units)	Float

2.3 Data Quality Observations

Missing Values

Source Code:

```
#detect missing values
df.isnull().sum()
```

Output:

	0
TV	0
radio	0
newspaper	0
sales	0

dtype: int64

The Sales Advertising dataset does not contain missing values. However, missing values should always be checked before model training.

Duplicate Values

Duplicate records should be identified and removed if present.

Outliers

Some advertising expenditure values may be unusually high or low and should be analyzed before training the model.

Data Type Verification

All columns are numerical (float) and suitable for Multiple Linear Regression.

3. Data Preparation

3.1 Data Cleaning

Performed the following preprocessing steps:

- Loaded the dataset.
- Checked for missing values.
- Removed duplicate records.
- Verified data types.
- Selected the required feature columns.
- Prepared the dataset for model training.

3.2 Feature Engineering

Independent Variables (X)

- TV Advertising Budget
- Radio Advertising Budget
- Newspaper Advertising Budget

Dependent Variable (Y)

Sales

3.3 Train-Test Split

Training Data = **80%**

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}") Output:
```

```
X_train shape: (160, 3)
X_test shape: (40, 3)
y_train shape: (160,)
y_test shape: (40,)
```

Testing Data = **20%**

Random State = **42**

4. Modelling

4.1 Model Selection

#create the model Source

Code: model =

LinearRegression()

model.fit(X_train, y_train)

Output:

#train the model

y_pred = model.predict(X_test)

#generate predictions for training and testing datasets using the trained model
y_train_pred = model.predict(X_train) y_test_pred = model.predict(X_test) print("Predictions generated")

Output:
Predictions generated

Algorithm Used

Multiple Linear Regression

Reason

Multiple Linear Regression predicts a dependent variable using **multiple independent variables** by fitting the best regression equation.

Regression Equation

$$\text{Sales} = \beta_0 + \beta_1(\text{TV}) + \beta_2(\text{Radio}) + \beta_3(\text{Newspaper})$$

where:

- β_0 = Intercept
- β_1 = Coefficient for TV
- β_2 = Coefficient for Radio
- β_3 = Coefficient for Newspaper

4.2 Model Training

Steps

1. Import required libraries.
2. Load the Sales Advertising dataset.
3. Check and clean the dataset.
4. Select TV, Radio, and Newspaper as input features (X).
5. Select Sales as the target variable (Y).
6. Split the dataset into training and testing sets.
7. Train the Multiple Linear Regression model.
8. Predict Sales values for the testing dataset.

4.3 Model Evaluation & Interpretation

Performance Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Square Error (RMSE)
- R² Score

Interpretation

- Lower MAE indicates more accurate predictions.
- Lower MSE indicates fewer squared prediction errors.
- Lower RMSE indicates lower prediction error. Higher R² Score indicates that the model explains more variation in Sales and provides a better fit.

5. Evaluation

5.1 Appropriate Metrics Selection

The following metrics are used:

MAE

Measures the average absolute prediction error.

MSE

Measures the average squared prediction error.

RMSE

Square root of MSE; represents prediction error in the same unit as Sales.

R² Score

Measures how well the independent variables explain the variation in Sales.

An ideal R² Score is close to 1.

#evaluate the model performace on testing data using metrics

```
Source Code: mae = mean_absolute_error(y_test, y_pred) mse =  
mean_squared_error(y_test, y_pred) rmse = np.sqrt(mse) r2 =  
r2_score(y_test, y_pred)
```

```
print(f"Mean Absolute Error (MAE): {mae:.2f}")  
print(f"Mean Squared Error (MSE): {mse:.2f}")  
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")  
print(f"R-squared (R2): {r2:.2f}") Output:
```

```
Mean Absolute Error (MAE): 1.46  
Mean Squared Error (MSE): 3.17  
Root Mean Squared Error (RMSE): 1.78  
R-squared (R2): 0.90
```

#evaluate the model using metrics for traning the data Source Code:

```
train_mae = mean_absolute_error(y_train, y_train_pred) train_mse =  
mean_squared_error(y_train, y_train_pred) train_rmse =  
np.sqrt(train_mse) train_r2 = r2_score(y_train, y_train_pred)  
print(f"Training Data - Mean Absolute Error (MAE): {train_mae:.2f}")  
print(f"Training Data - Mean Squared Error (MSE): {train_mse:.2f}")  
print(f"Training Data - Root Mean Squared Error (RMSE):  
{train_rmse:.2f}")  
print(f"Training Data - R-squared (R2): {train_r2:.2f}")
```

Output:

```
Training Data - Mean Absolute Error (MAE): 1.20  
Training Data - Mean Squared Error (MSE): 2.71  
Training Data - Root Mean Squared Error (RMSE): 1.64  
Training Data - R-squared (R2): 0.90
```

#slope Source

Code:

```
slope = model.coef_  
print("Slope:", slope) Output:
```

```
Slope: [0.04472952 0.18919505 0.00276111]
```

```
#intercept intercept =  
model.intercept_  
print("Intercept:", intercept)  
Output:
```

```
Intercept: 2.979067338122629
```

5.2 Interpretation

The Multiple Linear Regression model predicts **Sales** using **TV, Radio, and Newspaper advertising expenditures**.

If the **R² Score** is high and **MAE, MSE, and RMSE** are low, the model performs well and provides reliable sales predictions.

6. Conclusion

The **Sales Advertising Dataset** was successfully analyzed using **Multiple Linear Regression**.

The model learned the relationship between **TV, Radio, and Newspaper advertising expenditures** and **Sales**.

The developed model can help businesses forecast product sales, optimize advertising budgets, and improve marketing strategies.

7. Appendix:

Source Code:

```
# Visualize predictions vs. actual values  
plt.figure(figsize=(10, 6))  
sns.scatterplot(x=y_test, y=y_pred)  
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',  
lw=2) # Line for perfect prediction  
plt.xlabel('Actual Sales')  
plt.ylabel('Predicted Sales')  
plt.title('Actual vs. Predicted Sales')  
plt.grid(True)  
plt.show()
```

OutPut:

